



Universidad
Nacional
de Loja

Universidad Nacional de Loja

Facultad de la Energía, las Industrias y los Recursos Naturales no Renovables

Carrera de Ingeniería en Telecomunicaciones

Sistema de Monitoreo de Condiciones Ambientales en Aulas mediante Red de
Sensores Inalámbricos para el Smart Campus UNL

MANUAL DEL PROGRAMADOR

AUTORA:

María Andreina Montaña Rojas

DIRECTOR:

Ing. Christian Campoverde

Loja – Ecuador

2026

Educamos para **Transformar**

Tabla de Contenido

1.	Introducción	1
2.	Requisitos del Sistema	1
2.1	Requisitos de hardware	1
2.2	Requisitos de software	2
3.	Arquitectura del Software	3
3.1	Organización del repositorio del proyecto	3
3.2	Arquitectura general del sistema	5
3.3	Flujo de comunicación y de datos	6
4.	Instalación y Configuración del Sistema	7
4.1	Configuración del entorno de firmware	7
4.2	Despliegue del backend desde el repositorio	9
4.3	Componentes del backend.....	13
5.	Backend: Estructura y Modelo de Datos	14
5.1	Aplicación "sensores"	14
5.2	Modelo de datos: MedicionAmbiental.....	16
5.3	Formato de intercambio de datos (JSON).....	17
5.4	Verificación de la base de datos	18
6.	Firmware de los nodos sensores	19
6.1	Organización del firmware.....	19
6.2	Funcionamiento general del firmware.....	20
6.3	Configuración de los parámetros del firmware	21
6.4	Lectura de sensores	23
6.5	Firmware del gateway de pruebas.....	25
7.	Solución de Problemas.....	27
8.	Mantenimiento, Escalabilidad y Seguridad	28

8.1	Mantenimiento	28
8.2	Escalabilidad del sistema	29
8.3	Seguridad y respaldo	29

1. Introducción

El presente Manual del Programador tiene como finalidad proporcionar la documentación técnica del Sistema de Monitoreo de Condiciones Ambientales en Aulas, desarrollado como proyecto de titulación de la carrera de Ingeniería en Telecomunicaciones, en el marco de la iniciativa Smart Campus de la Universidad Nacional de Loja (UNL). En este documento se describe la arquitectura del software, la organización del repositorio, el modelo de datos, el firmware de los nodos sensores y los procedimientos necesarios para la instalación, configuración y mantenimiento del sistema.

Este manual está dirigido a desarrolladores, técnicos y evaluadores que requieran comprender la implementación interna del proyecto para realizar tareas de mantenimiento, actualización o ampliación de sus funcionalidades. Se asume que el lector posee conocimientos básicos de programación en C++ (Arduino) y Python (Django); por ello, el contenido se centra en la estructura y funcionamiento de la solución desarrollada, sin abordar conceptos generales de dichos lenguajes.

El sistema está conformado por tres componentes principales: los nodos sensores basados en la placa Heltec WiFi LoRa 32 V3, un gateway de pruebas encargado de recibir las transmisiones LoRa y reenviar la información al servidor, y un backend desarrollado con el framework Django, responsable de procesar, almacenar y presentar las mediciones ambientales mediante un dashboard web.

El contenido de este manual complementa al Manual de Usuario, el cual está orientado a la operación del sistema una vez desplegado, mientras que el presente documento proporciona la información necesaria para comprender su estructura interna y facilitar futuras tareas de desarrollo y mantenimiento.

2. Requisitos del Sistema

2.1 Requisitos de hardware

Tabla 1

Componentes de hardware requeridos (2 Nodos)

Cantidad	Componente	Descripción técnica
2	Heltec WiFi LoRa 32 V3	Microcontrolador principal (ESP32-S3) con módulo LoRa y pantalla OLED integrados; uno por cada nodo sensor (Aula 1 y Aula 2).

Cantidad	Componente	Descripción técnica
2	Sensor SHT31	Sensor de temperatura y humedad relativa, comunicación I2C.
2	Sensor MH-Z19C	Sensor de concentración de CO ₂ , comunicación UART.
2	Sensor BH1750	Sensor de iluminación, comunicación I2C.
2	Micrófono INMP441	Micrófono digital para nivel de ruido, comunicación I2S.
2	Sensor PIR HC-SR501	Sensor de presencia / movimiento, salida digital.
1	Servidor (PC o equipo dedicado)	Equipo con Python y PostgreSQL en ejecución, donde se despliega el backend Django.

2.2 Requisitos de software

Tabla 2

Herramientas y librerías de software utilizadas

Herramienta / Librería	Descripción y propósito
Arduino IDE 2.3.7 o superior	Entorno de desarrollo integrado utilizado para compilar y cargar el firmware en los nodos sensores.
ESP32 by Espressif Systems	Paquete de soporte para placas ESP32, necesario para compilar el firmware de la Heltec WiFi LoRa 32 V3.
heltec_unofficial	Librería utilizada para controlar el módulo LoRa SX1262 y la pantalla OLED integrados en la placa Heltec WiFi LoRa 32 V3.
Wire	Librería para la comunicación mediante el protocolo I2C con los sensores ambientales.
Adafruit_SHT31	Librería para la lectura del sensor de temperatura y humedad SHT31.
BH1750	Librería para la lectura del sensor de iluminación BH1750.
math.h	Biblioteca estándar de C/C++ utilizada para realizar operaciones matemáticas durante el procesamiento del nivel de ruido obtenido por el micrófono INMP441.

Herramienta / Librería	Descripción y propósito
driver/i2s (ESP-IDF)	Controlador utilizado para la comunicación I2S con el micrófono digital INMP441.
WiFi	Librería utilizada por el gateway para establecer la conexión con la red inalámbrica.
HTTPClient	Librería utilizada por el gateway para enviar las mediciones al backend mediante solicitudes HTTP.
Python 3.11 o superior	Entorno de ejecución del backend desarrollado en Django.
Django 5.x	Framework web utilizado para implementar el backend del sistema.
Django REST Framework	Framework utilizado para desarrollar la API REST del sistema.
PostgreSQL 14 o superior	Sistema gestor de base de datos utilizado para almacenar las mediciones ambientales.
python-dotenv	Librería para cargar las variables de entorno definidas en el archivo .env.
Git	Sistema de control de versiones utilizado para gestionar el código fuente del proyecto.

Nota. Se asume que el desarrollador dispone de Python, PostgreSQL y Git previamente instalados y configurados en el equipo de desarrollo. Este manual se centra en la configuración y despliegue del proyecto, por lo que no aborda la instalación general de dichas herramientas.

3. Arquitectura del Software

Este capítulo presenta la organización del repositorio del proyecto, la arquitectura lógica del sistema y el flujo de comunicación entre sus componentes, proporcionando una visión general de la estructura del software antes de abordar su instalación y funcionamiento interno.

3.1 Organización del repositorio del proyecto

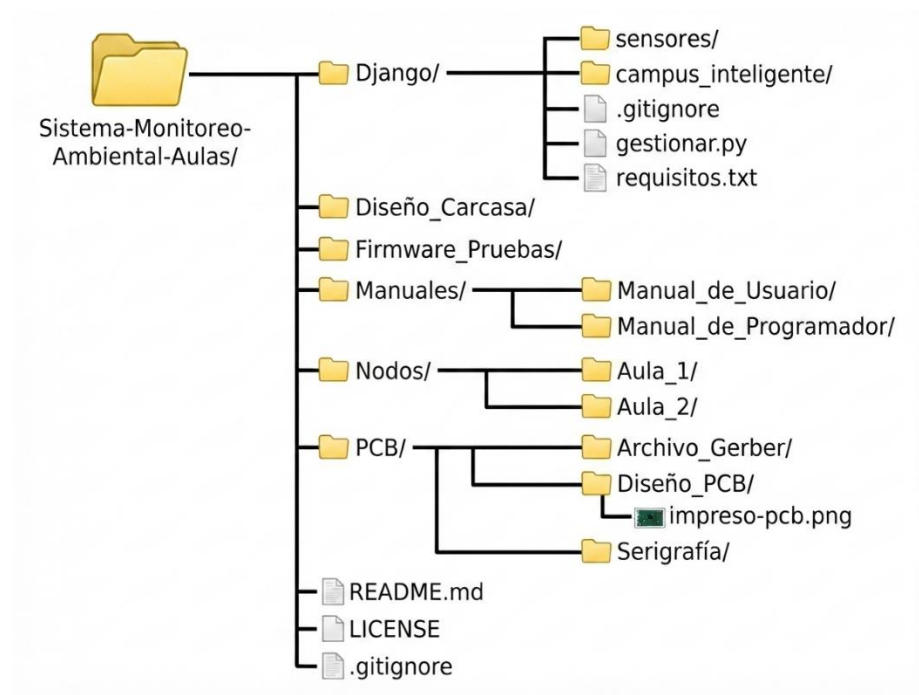
El proyecto se organiza en un único repositorio Git, el cual integra el código fuente, los diseños de hardware y la documentación técnica. La Tabla 3 resume la estructura principal de directorios utilizada para el desarrollo y mantenimiento del sistema.

Tabla 3*Estructura de carpetas del repositorio del proyecto*

Carpeta	Contenido
Nodos/Aula_1, Nodos/Aula_2	Contiene el firmware de los nodos sensores desarrollado en Arduino. Ambos proyectos comparten la misma lógica de funcionamiento y únicamente difieren en los parámetros de configuración descritos en la Sección 6.9.
Firmware de pruebas	Contiene el firmware utilizado durante la etapa de pruebas para el nodo receptor encargado de recibir las tramas LoRa y reenviarlas al backend.
tarjeta de circuito impreso/ PCB	Incluye los archivos de diseño de la placa electrónica, los archivos Gerber para fabricación y la serigrafía utilizada en la PCB.
Diseño_Carcasa/	Contiene el modelo tridimensional de la carcasa diseñada para alojar y proteger el nodo sensor.
Django	Contiene el backend del sistema desarrollado en Django, incluyendo el proyecto smartcampus, la aplicación sensores, el archivo manage.py, las dependencias (requirements.txt) y los archivos de configuración necesarios para el despliegue.
Manuales	Contiene la documentación técnica del proyecto, incluyendo el Manual de Usuario y el Manual del Programador.
README.md, LICENCIA	Archivos con la descripción general del proyecto, instrucciones básicas de uso y la licencia del repositorio.

Figura 1

Estructura del proyecto



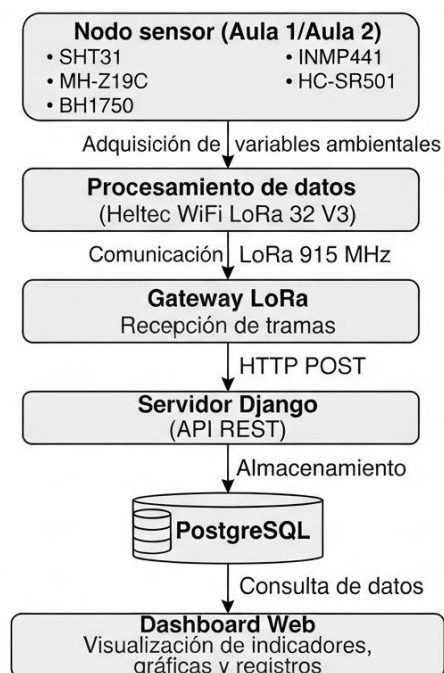
Nota. La figura presenta la estructura jerárquica del repositorio del proyecto, mostrando la organización de los principales directorios y archivos que conforman el sistema. Elaboración propia.

3.2 Arquitectura general del sistema

El sistema se organiza en cuatro componentes lógicos: los nodos sensores, el nodo gateway, el backend Django y el dashboard web. Los nodos sensores miden las variables ambientales y las transmiten mediante LoRa; el gateway recibe dichas transmisiones y las reenvía al backend mediante una petición HTTP; el backend valida, persiste y expone los datos a través de una API REST; y el dashboard consume esa API para presentar la información al usuario final.

Figura 2

Arquitectura general del sistema



Nota. El funcionamiento del sistema inicia con la adquisición de las variables ambientales mediante los sensores integrados en el nodo sensor. Posteriormente, la tarjeta Heltec WiFi LoRa 32 V3 procesa las mediciones y genera una trama de datos que es transmitida mediante tecnología LoRa hacia el gateway. Una vez recibida la información, el gateway envía las mediciones al servidor Django mediante una solicitud HTTP POST, donde la API REST procesa la información y la almacena en la base de datos PostgreSQL. Finalmente, el dashboard consulta los registros almacenados y presenta al usuario los indicadores, gráficas e historial de mediciones correspondientes a las variables monitoreadas. Elaboración propia.

3.3 Flujo de comunicación y de datos

A nivel de datos, cada medición atraviesa las siguientes etapas desde su captura hasta su visualización: adquisición en el nodo sensor, empaquetado en formato JSON, transmisión LoRa hacia el gateway, reenvío HTTP hacia el backend, validación y persistencia en PostgreSQL, y finalmente consulta y presentación en el dashboard. Esta secuencia constituye el ciclo de vida completo de un registro de MedicionAmbiental, y se detalla nuevamente, a nivel de código, en la sección 7.2.

Figura 3

Flujo de datos del sistema



4. Instalación y Configuración del Sistema

Esta sección describe la configuración necesaria para desplegar el sistema a partir del repositorio del proyecto, asumiendo que las herramientas de base (Arduino IDE, Python, PostgreSQL, Git) ya se encuentran instaladas en los equipos correspondientes.

4.1 Configuración del entorno de firmware

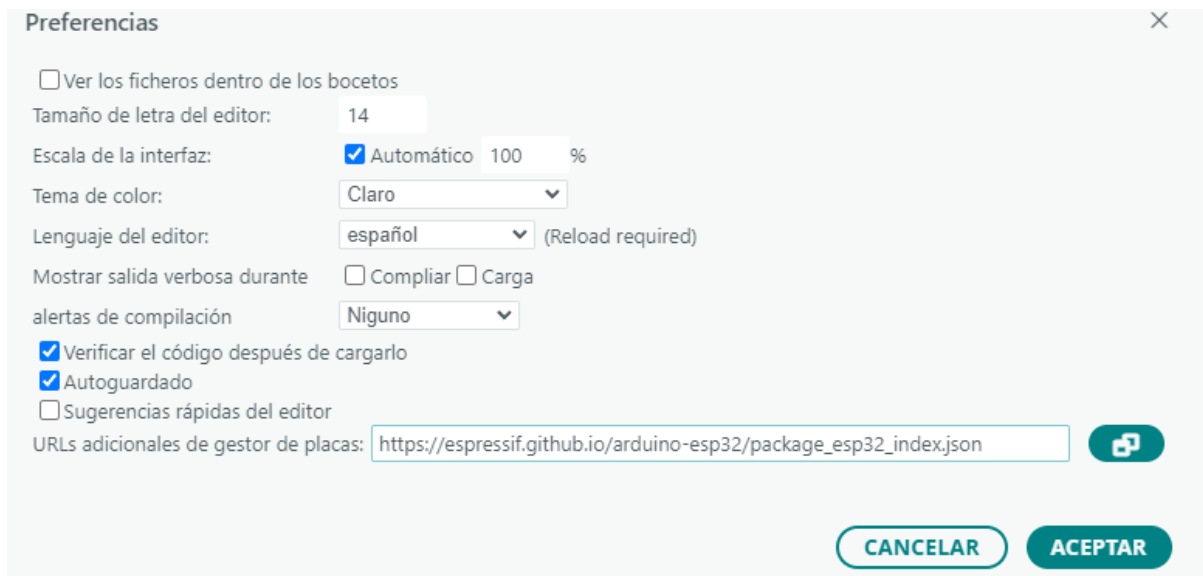
Antes de compilar el firmware de los nodos sensores es necesario preparar el entorno de desarrollo en Arduino IDE. Para ello se deben realizar los siguientes pasos:

Paso 1. Instalar el soporte para ESP32

En Arduino IDE acceder a Archivo → Preferencias y agregar la URL del gestor de tarjetas de Espressif Systems. En el campo "Gestor de URLs Adicionales de Tarjetas", añadir: https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json e instalar la versión compatible con la placa Heltec WiFi LoRa 32 V3.

Figura 4

Gestor de Tarjetas de Arduino IDE

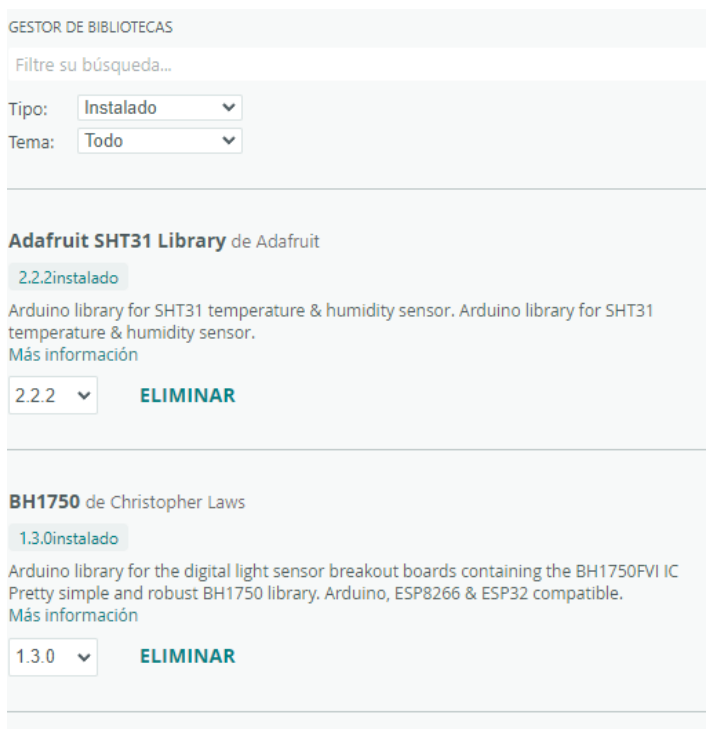


Paso 2. Instalar las librerías del proyecto

Desde el Gestor de Bibliotecas instalar las librerías indicadas en la Tabla 2, necesarias para el funcionamiento del firmware de los nodos.

Figura 5

Gestor de bibliotecas del Arduino IDE mostrando la instalación de librerías

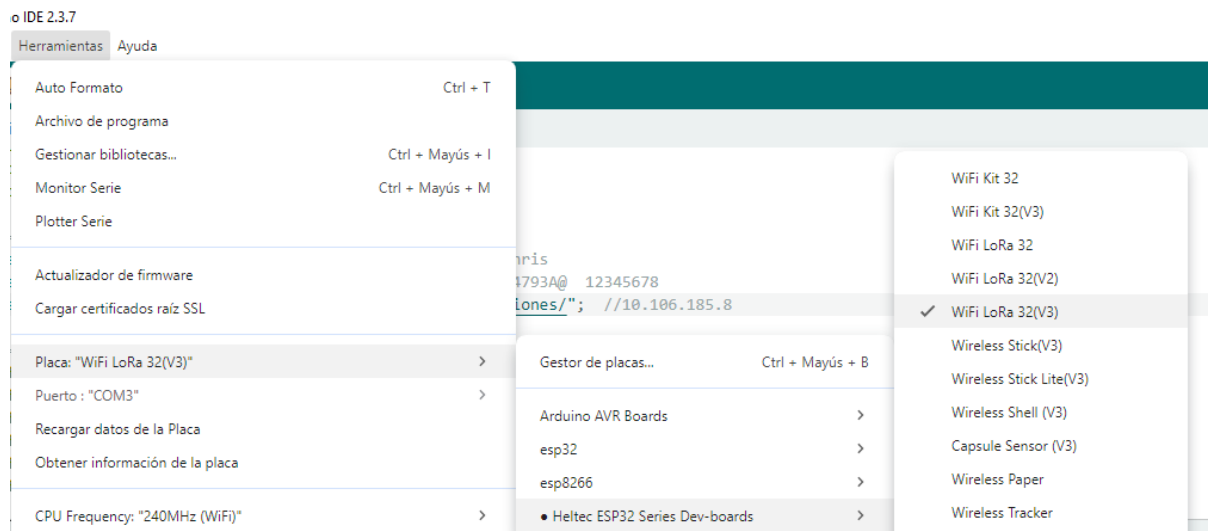


Paso 3. Seleccionar la placa y el puerto

Una vez instaladas las dependencias, seleccionar la placa Heltec WiFi LoRa 32 V3 desde el menú Herramientas → Placa, así como el puerto COM correspondiente al dispositivo conectado.

Figura 6

Gestor de tarjetas mostrando la instalación del paquete ESP32



4.2 Despliegue del backend desde el repositorio

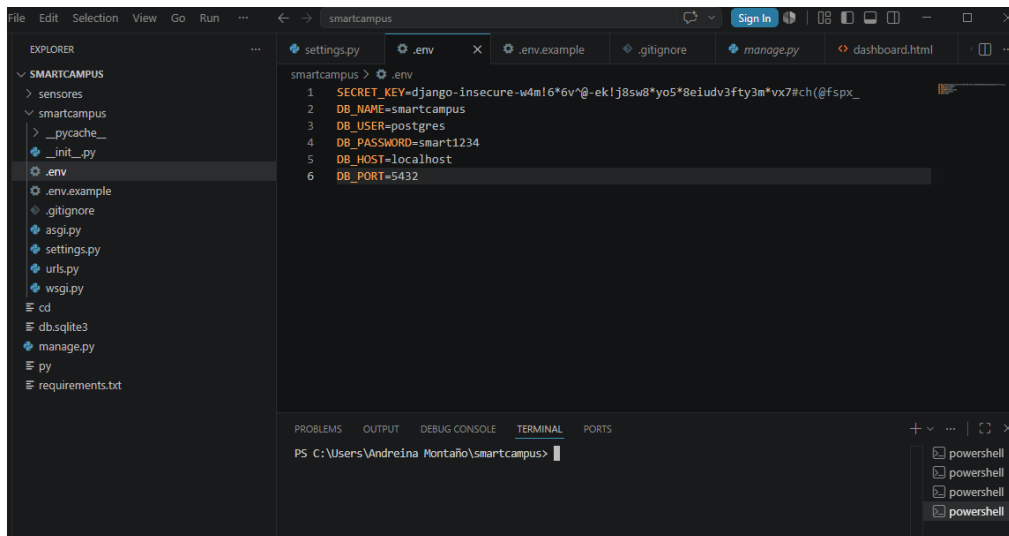
Una vez clonado el repositorio del proyecto, el despliegue del backend se realiza desde la carpeta Django, la cual contiene el proyecto desarrollado con el framework Django. Todos los comandos descritos en esta sección se ejecutan desde la terminal integrada de Visual Studio Code, utilizando como directorio de trabajo la carpeta raíz del proyecto.

Paso 1. Abrir el proyecto en Visual Studio Code

Abrir Visual Studio Code y seleccionar Archivo → Abrir carpeta, luego se debe elegir la carpeta Django del repositorio clonado. Una vez abierto el proyecto, acceder a Terminal → Nueva terminal para abrir la terminal integrada de Visual Studio Code. Verificar que la ruta de trabajo corresponda a la carpeta donde se encuentran los archivos manage.py, requirements.txt y las carpetas sensores y smartcampus.

Figura 7

Apertura del proyecto Django y terminal integrada de Visual Studio Code



Nota. La terminal debe ubicarse en la carpeta raíz del proyecto, donde se encuentra el archivo `manage.py`, ya que desde esta ubicación se ejecutan todos los comandos del backend.

Paso 2. Instalar las dependencias del proyecto

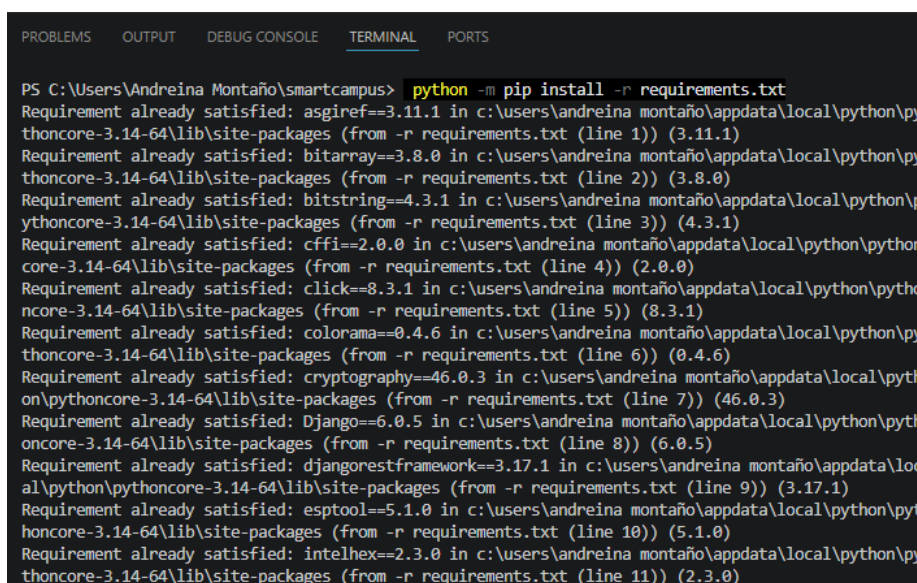
Una vez abierta la terminal integrada, ejecutar el siguiente comando:

`python -m pip install -r requirements.txt`

Este comando instala automáticamente todas las dependencias definidas para el proyecto, incluyendo Django, Django REST Framework, psycopg2-binary para la comunicación con PostgreSQL y python-dotenv para la gestión de variables de entorno.

Figura 8

Instalación de las dependencias del proyecto mediante el archivo `requirements.txt`



Paso 3. Configurar las variables de entorno

En la carpeta principal del proyecto se incluye el archivo **.env.example**, el cual contiene la estructura básica de las variables de configuración necesarias para ejecutar el sistema. Crear una copia de este archivo con el nombre **.env** y completar los valores correspondientes a la configuración local del servidor.

El archivo **.env** debe contener, al menos, las siguientes variables:

SECRET_KEY=...

DB_NAME=...

DB_USER=...

DB_PASSWORD=...

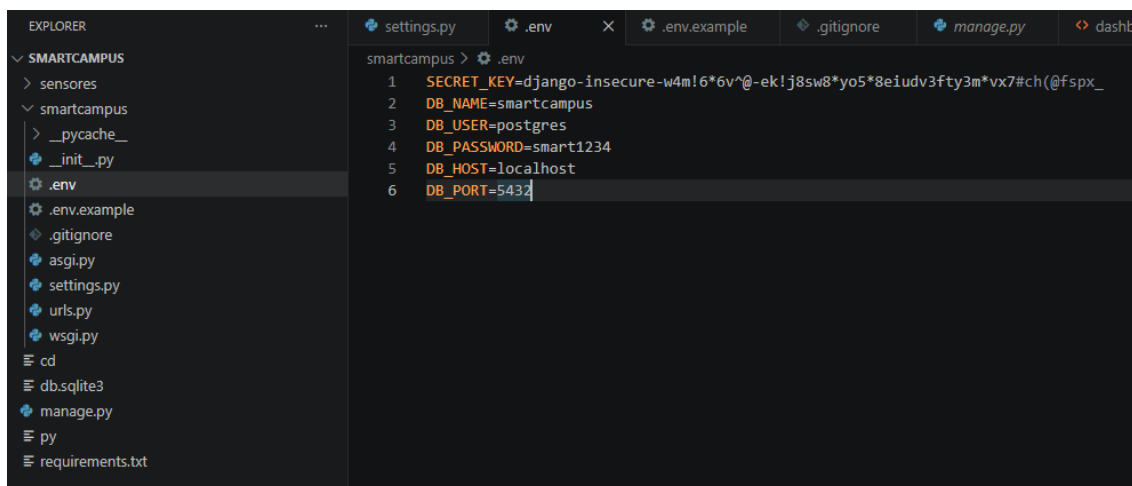
DB_HOST=localhost

DB_PORT=5432

El archivo **.env** no debe incorporarse al repositorio de GitHub, ya que almacena información sensible, como la clave secreta de Django y las credenciales de acceso a la base de datos.

Figura 9

Archivo .env configurado con los parámetros de conexión utilizados por el proyecto



Paso 4. Aplicar las migraciones de la base de datos

Con la terminal ubicada en la carpeta raíz del proyecto, ejecutar el siguiente comando:

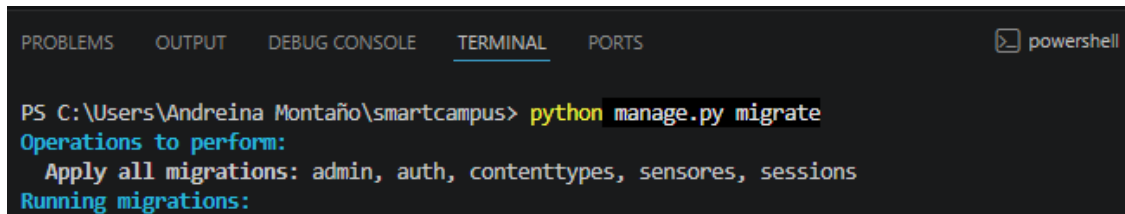
python manage.py migrate

Este procedimiento crea, en la base de datos PostgreSQL, todas las tablas requeridas por el proyecto, incluyendo las correspondientes al modelo **MedicionAmbiental**, así como las tablas internas utilizadas por Django para la administración de usuarios, sesiones y permisos.

Cuando las migraciones ya hayan sido ejecutadas previamente, el sistema mostrará el mensaje: No migrations to apply.

Figura 10

Ejecución satisfactoria del comando



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell
PS C:\Users\Andreina Montaña\smartcampus> python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sensores, sessions
Running migrations:
```

Paso 5. Iniciar el servidor de desarrollo

Finalmente, iniciar el servidor del backend mediante el siguiente comando: **python manage.py runserver**

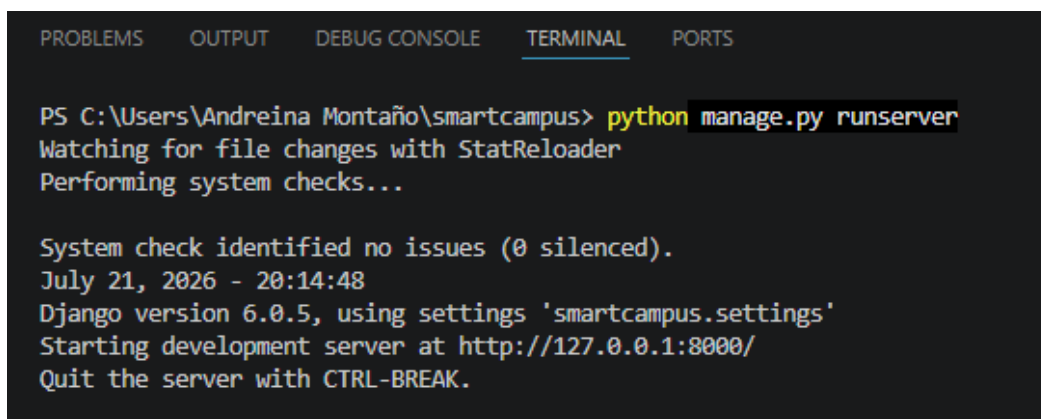
Si la ejecución es correcta, Django iniciará el servidor de desarrollo y mostrará la dirección desde la cual será posible acceder al sistema.

Por defecto, el backend estará disponible en: <http://127.0.0.1:8000/>

Desde esta dirección se podrá acceder tanto al dashboard del sistema como a los servicios REST implementados para la recepción y consulta de las mediciones ambientales.

Figura 11

Inicio del servidor de desarrollo mediante el comando



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Andreina Montaña\smartcampus> python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
July 21, 2026 - 20:14:48
Django version 6.0.5, using settings 'smartcampus.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

Nota. Mientras la terminal permanezca abierta, el servidor continuará ejecutándose y el backend estará disponible para recibir las mediciones enviadas por el nodo receptor utilizado durante las pruebas del sistema.

4.3 Componentes del backend

El backend del sistema fue desarrollado utilizando el framework Django y constituye el componente encargado de recibir, validar, almacenar y presentar la información enviada por los nodos sensores. Su estructura está organizada en cuatro componentes funcionales principales, los cuales trabajan de forma integrada para garantizar el procesamiento de las mediciones ambientales.

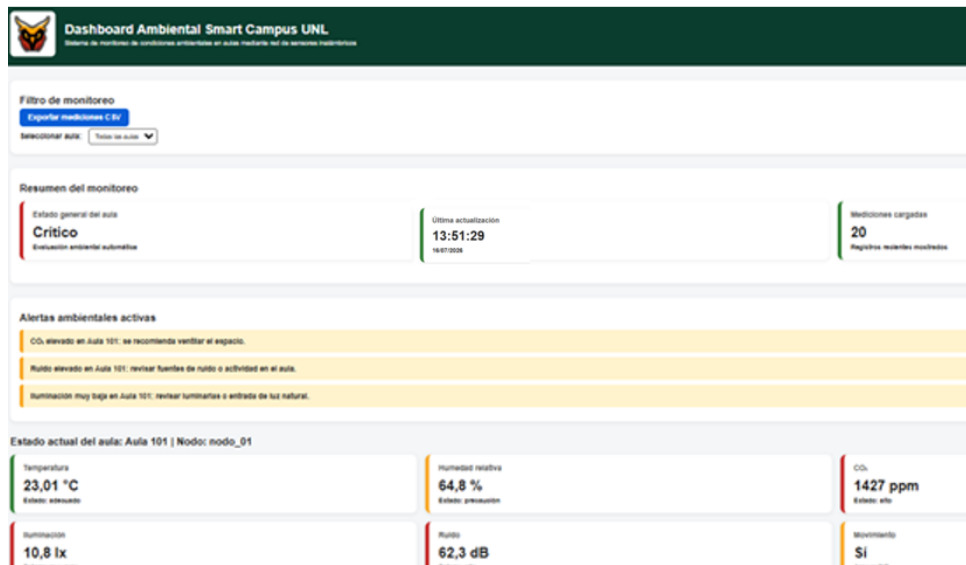
Tabla 4

Componentes del backend

Componentes	Función dentro del proyecto
API REST	Constituye la interfaz de comunicación entre el gateway de pruebas y el servidor. Recibe las solicitudes HTTP POST que contienen las mediciones ambientales en formato JSON, valida la información recibida y la procesa antes de almacenarla en la base de datos.
Base de datos PostgreSQL	Utilizado para almacenar de forma persistente todas las mediciones ambientales recibidas. El acceso a la base de datos es gestionado mediante el ORM de Django, permitiendo manipular la información sin escribir consultas SQL de forma manual.
Dashboard web	Permite visualizar en tiempo real las variables ambientales registradas por los nodos sensores, así como consultar el historial de mediciones de cada aula mediante tablas e indicadores gráficos.
Panel de administración de Django	Django incorpora un panel administrativo accesible mediante la ruta /admin/ , desde el cual es posible consultar, agregar, modificar o eliminar registros almacenados en la base de datos. Esta herramienta facilita las tareas de administración y verificación del sistema durante las etapas de desarrollo y mantenimiento.

Figura 12

Dashboard y componentes principales del backend



Nota. Captura de la interfaz principal del sistema web desarrollado en Django, donde se visualizan las variables ambientales registradas por los nodos sensores y la información obtenida desde la base de datos PostgreSQL. Elaboración propia.

5. Backend: Estructura y Modelo de Datos

El backend del sistema fue desarrollado utilizando el framework Django, siguiendo una arquitectura modular basada en aplicaciones. La lógica principal se concentra en la aplicación **sensores**, responsable de recibir las mediciones provenientes del gateway de pruebas, validar la información recibida, almacenarla en la base de datos PostgreSQL y proporcionar los datos necesarios para su visualización en el dashboard web. En este capítulo se describe la organización de la aplicación, el modelo de datos implementado y el formato de intercambio de información utilizado entre los diferentes componentes del sistema.

5.1 Aplicación "sensores"

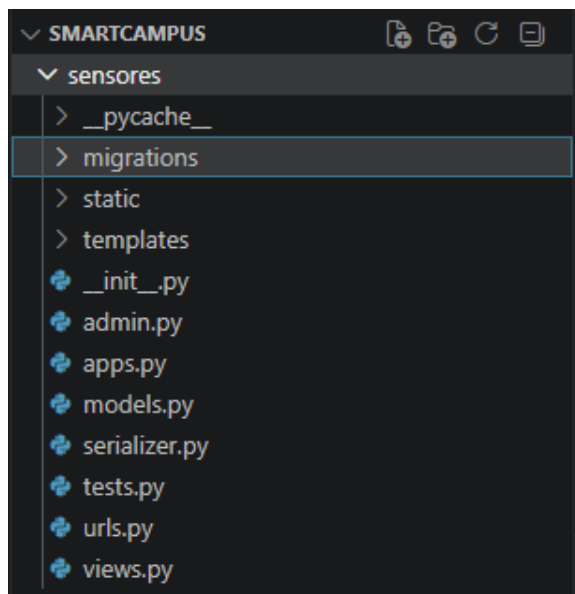
La aplicación **sensores** constituye el núcleo funcional del backend. En ella se implementan el modelo de datos, la lógica de procesamiento, la serialización de la información, las vistas de la API REST y la interfaz web utilizada para la visualización de las mediciones ambientales. La organización de sus principales archivos se presenta en la Tabla 5.

Tabla 5*Archivos principales de la aplicación sensores*

Archivo	Función dentro del proyecto
models.py	Define el modelo MedicionAmbiental , el cual representa la estructura de las mediciones almacenadas en PostgreSQL mediante el ORM de Django.
serializers.py	Implementa la serialización y validación de los datos recibidos en formato JSON antes de su almacenamiento en la base de datos.
views.py	Contiene la lógica de las vistas encargadas de recibir las solicitudes HTTP provenientes del gateway, procesar las consultas del dashboard y generar las respuestas de la API REST.
urls.py	Define las rutas de acceso a los servicios REST y a las vistas del sistema, asociando cada URL con su correspondiente función o clase.
admin.py	Registra el modelo MedicionAmbiental en el panel administrativo de Django para facilitar la gestión de los registros.
templates/sensores/dashboard.html	Implementa la interfaz gráfica principal del sistema para la visualización de las mediciones ambientales.

Figura 13

Estructura de la aplicación sensores del backend



5.2 Modelo de datos: MedicionAmbiental

El sistema utiliza un único modelo denominado **MedicionAmbiental**, el cual representa cada registro recibido desde los nodos sensores. Este modelo fue implementado mediante el ORM (Object Relational Mapping) de Django, permitiendo que cada objeto creado en la aplicación sea almacenado automáticamente como un registro dentro de la base de datos PostgreSQL.

Cada instancia del modelo almacena la información correspondiente a una medición ambiental, incluyendo la identificación del nodo emisor, el aula de origen, las variables ambientales registradas y la fecha y hora en que el servidor recibió la información.

La estructura del modelo se presenta en la Tabla 6.

Tabla 6

Campos del modelo MedicionAmbiental

Campo	Tipo de dato	Descripción
id	AutoField	Identificador único generado automáticamente por Django (clave primaria).

Campo	Tipo de dato	Descripción
nodo	CharField	Identificador del nodo sensor que realizó la medición.
aula	CharField	Nombre del aula donde se registró la medición.
temperatura	FloatField	Temperatura ambiente expresada en grados Celsius (°C).
humedad	FloatField	Humedad relativa del ambiente (%).
co2	IntegerField	Concentración de dióxido de carbono (ppm).
iluminacion	FloatField	Nivel de iluminación, en lux.
ruido	FloatField	Nivel sonoro relativo, en dB.
movimiento	BooleanField	Estado de detección de presencia.

Nota. El campo fecha_hora no es enviado por los nodos sensores dentro del paquete JSON. Este valor es generado automáticamente por Django en el momento en que la medición es almacenada en la base de datos, permitiendo registrar el instante exacto de recepción por parte del servidor.

5.3 Formato de intercambio de datos (JSON)

La comunicación entre los nodos sensores, el gateway utilizado durante la etapa de pruebas y el backend se realiza mediante mensajes en formato **JSON**. Este formato permite transmitir de manera estructurada las variables ambientales medidas por los sensores, facilitando su procesamiento por la API REST desarrollada en Django.

Cada paquete enviado contiene la identificación del nodo, el aula de origen y las variables ambientales registradas en el instante de la medición. La correspondencia entre cada campo del paquete y el modelo de datos implementado en el backend es directa.

Tabla 7

Campos del paquete JSON transmitido por los nodos

Campo	Tipo de dato	Descripción
nodo	string	Identificador del nodo sensor que realizó la medición.
aula	string	Aula donde se encuentra instalado el nodo.
temperatura	number (float)	Temperatura ambiente (°C).
humedad	number (float)	Humedad relativa (%).
co2	number (integer)	Concentración de dióxido de carbono (ppm).
iluminacion	number (float)	Nivel de iluminación (lux).
ruido	number (float)	Nivel sonoro relativo en dB.
movimiento	boolean	true si se detectó presencia; false en caso contrario.

El siguiente ejemplo muestra la estructura de un paquete transmitido por un nodo sensor.

Figura 14

Ejemplo de paquete JSON enviado al backend

```
</> JSON

{
  "nodo": "nodo_01",
  "aula": "Aula 1",
  "temperatura": 23.6,
  "humedad": 56.2,
  "co2": 640,
  "iluminacion": 185.4,
  "ruido": 42.7,
  "movimiento": true
}
```

Nota: El campo **fecha_hora** no forma parte del paquete JSON, ya que es generado automáticamente por el backend al momento de almacenar la medición en la base de datos.

5.4 Verificación de la base de datos

Una vez que el backend recibe las mediciones enviadas por el gateway de pruebas, la información es almacenada en la base de datos PostgreSQL mediante el modelo MedicionAmbiental. Para verificar que el almacenamiento se realiza correctamente, el desarrollador puede utilizar **pgAdmin 4**, herramienta que permite consultar las bases de datos y visualizar los registros almacenados.

Desde el esquema public es posible acceder a la tabla sensores_medicionambiental, donde se registran las mediciones recibidas por el sistema. Cada registro corresponde a una transmisión realizada por un nodo sensor e incluye la identificación del nodo, el aula de origen, las variables ambientales registradas y la fecha de recepción generada automáticamente por Django.

Figura 15

Visualización de la tabla sensores_medicionambiental en pgAdmin 4

	id [PK] bigint	temperatura double precision	humedad double precision	co2 integer	iluminacion double precision	ruido double precision	movimiento boolean	fecha_hora timestamp with time zone
1	3915	20.69	68.13	506	24.2	30	true	2026-07-16 13:55:00.000000+00
2	3914	20.69	67.95	512	25	30	true	2026-07-16 13:55:00.000000+00
3	3913	20.69	67.89	529	27.5	30	true	2026-07-16 13:55:00.000000+00
4	3912	20.72	67.8	544	26.7	30	true	2026-07-16 13:55:00.000000+00
5	3911	20.7	67.92	559	28.3	30	true	2026-07-16 13:55:00.000000+00
6	3910	20.7	67.98	559	25.8	30	true	2026-07-16 13:55:00.000000+00

Nota. La consulta de los registros desde pgAdmin 4 permite verificar que las mediciones enviadas por los nodos sensores fueron almacenadas correctamente en PostgreSQL.

6. Firmware de los nodos sensores

El firmware de los nodos sensores fue desarrollado en el entorno Arduino IDE utilizando el lenguaje C++. Su función es adquirir las mediciones de los sensores ambientales instalados en cada aula, procesar la información obtenida y transmitirla mediante tecnología LoRa hacia el nodo receptor empleado durante la etapa de validación del sistema.

El proyecto incluye dos firmwares principales, correspondientes a los nodos Aula 1 y Aula 2, los cuales comparten la misma estructura de programación y únicamente difieren en los parámetros de identificación y temporización definidos para cada nodo.

6.1 Organización del firmware

El código fuente del firmware se encuentra organizado dentro de la carpeta **Nodos** del repositorio, donde cada subcarpeta contiene el programa correspondiente a un nodo sensor.

Figura 16

Organización del firmware de los nodos

Carpeta	Descripción
Nodos/Aula_1	Firmware del nodo instalado en el Aula 1.
Nodos/Aula_2	Firmware del nodo instalado en el Aula 2.
Firmware_Pruebas	Firmware del nodo receptor utilizado durante la etapa de validación del sistema.

Nota. El repositorio contiene un firmware para cada nodo sensor y un firmware adicional correspondiente al gateway de prueba utilizado durante la etapa de validación del sistema.

Los programas correspondientes a los nodos Aula 1 y Aula 2 comparten la misma estructura de funcionamiento. Las diferencias entre ambos se limitan a los parámetros de configuración que identifican cada nodo dentro del sistema, evitando mantener múltiples versiones independientes del código fuente.

6.2 Funcionamiento general del firmware

El firmware implementado en los nodos sensores ejecuta de forma continua un ciclo de adquisición, procesamiento y transmisión de datos ambientales. Una vez energizado el dispositivo, el programa inicializa los periféricos de la placa Heltec WiFi LoRa 32 V3, establece la comunicación con los sensores y configura el módulo de radio LoRa. Posteriormente, entra en un ciclo de ejecución permanente en el que realiza la lectura de las variables ambientales, actualiza la información mostrada en la pantalla OLED y transmite periódicamente las mediciones hacia el nodo receptor utilizado durante la validación del sistema.

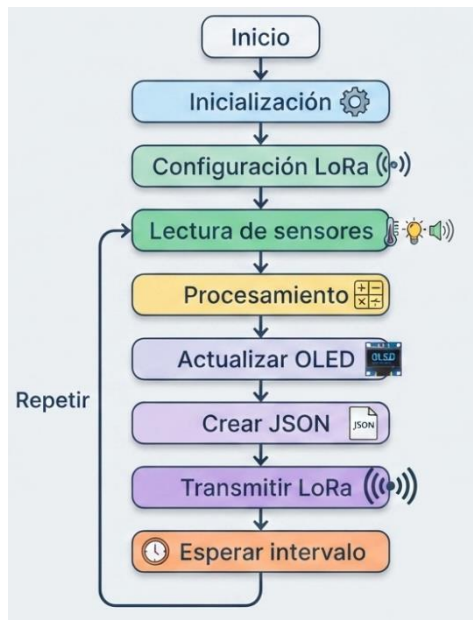
El funcionamiento general del firmware se resume en las siguientes etapas:

1. Inicialización de la placa y de los sensores.
2. Configuración del módulo de comunicación LoRa.
3. Adquisición de las variables ambientales.
4. Procesamiento de las lecturas obtenidas.
5. Visualización de la información en la pantalla OLED.
6. Construcción del paquete de datos en formato JSON.

7. Transmisión de la información mediante tecnología LoRa.
8. Repetición continua del ciclo de monitoreo.

Figura 17

Flujo general de funcionamiento del firmware de los nodos sensores



Nota. El firmware ejecuta de forma continua un ciclo de adquisición, procesamiento y transmisión de datos mientras el nodo permanezca en funcionamiento. Elaboración propia.

6.3 Configuración de los parámetros del firmware

Antes de compilar y cargar el firmware en cada nodo sensor, es necesario verificar los parámetros de configuración definidos dentro del código fuente. Estos parámetros permiten identificar cada dispositivo dentro de la red, establecer la comunicación mediante LoRa y definir el intervalo de transmisión de las mediciones.

La **Tabla 8** presenta los principales parámetros configurables del firmware.

Tabla 8

Parámetros de configuración del firmware

Parámetro	Valor	Descripción
Frecuencia LoRa	915.0 MHz	Frecuencia de operación utilizada por los nodos para la comunicación inalámbrica. Debe coincidir en todos

Parámetro	Valor	Descripción
		los dispositivos de la red. (Banda ISM para Ecuador)
NODO_ID	Nodo_01,nodo__02...nodo_N	Identificador único asignado al nodo sensor. Este valor se incluye en cada paquete de datos transmitido al backend.
AULA_ID	Aula_1, Aula_2...Aula_N	Identifica el aula donde se encuentra instalado el nodo sensor.
Ancho de banda (BW)	125 kHz	Ancho de banda configurado para la comunicación LoRa.
Spreading Factor (SF)	7	Factor de expansión utilizado durante la transmisión LoRa.
Coding Rate (CR)	4/5	Tasa de corrección de errores utilizada durante la transmisión.
Sync Word	0x12	Identificador de la red LoRa empleado para permitir únicamente la comunicación entre dispositivos configurados con el mismo valor.
Potencia de transmisión	14 dBm	Potencia de salida utilizada por el módulo LoRa durante la transmisión de los datos.

Los parámetros relacionados con la comunicación LoRa deben mantenerse iguales en todos los dispositivos que participan en la comunicación inalámbrica. En cambio, los parámetros **NODO_ID**, **AULA_ID** e **INTERVALO_ENVIO** deben configurarse de forma específica para cada nodo sensor.

Tabla 9

Configuración utilizada en los nodos del proyecto

Nodo	NODO_ID	AULA_ID	Intervalo de envío
Aula 1	nodo_01	Aula 1	15 000 ms
Aula 2	nodo_02	Aula 2	17 000 ms

Nota: Los diferentes intervalos de transmisión reducen la probabilidad de que ambos nodos transmitan simultáneamente, disminuyendo las posibles colisiones durante la comunicación LoRa.

Figura 18

Parámetros de configuración del firmware de los nodos sensores

```
// ===== CONFIGURACION LORA =====  
// Debe coincidir exactamente con la configuracion del gateway.  
#define FRECUENCIA_LORA 915.0 // MHz (banda ISM usada en la region)  
#define ANCHO_BANDA 125.0 // kHz  
#define SPREADING 7 // Spreading Factor (SF7)  
#define CODING_RATE 5 // Coding Rate 4/5  
#define SYNC_WORD 0x12 // Palabra de sincronizacion de la red privada  
#define POTENCIA_TX 14 // dBm  
  
// Identificadores del nodo: UNICO valor que cambia entre Aula 1 y Aula 2.  
const char* NODO_ID = "nodo_02";  
const char* AULA_ID = "Aula 2";
```

Nota. Fragmento del código fuente donde se definen los parámetros de identificación del nodo y los valores de configuración utilizados durante la comunicación LoRa.

6.4 Lectura de sensores

El firmware realiza la adquisición periódica de las variables ambientales mediante los sensores conectados a la placa **Heltec WiFi LoRa 32 V3**. Cada sensor utiliza una interfaz de comunicación específica y proporciona una variable que posteriormente es incorporada al paquete JSON transmitido al backend.

La **Tabla 10** resume los sensores utilizados por el sistema y su función dentro del proceso de monitoreo.

Tabla 10

Sensores implementados en el sistema

Sensor	Variable medida	Interfaz de comunicación	Función dentro del sistema
SHT31	Temperatura y humedad relativa	I2C	Mide las condiciones térmicas del aula.
BH1750	Iluminación	I2C	Registra el nivel de iluminación del aula en lux.
MH-Z19C	Concentración de CO ₂	UART	Determina la concentración de dióxido de carbono presente en el ambiente.
HC-SR501 (PIR)	Movimiento	Digital	Detecta la presencia de movimiento dentro del aula.

Sensor	Variable medida	Interfaz de comunicación	Función dentro del sistema
INMP441	Nivel de ruido	I2S	Captura la señal de audio para estimar el nivel de ruido ambiental.

Cada ciclo de ejecución del firmware consulta los sensores, procesa las lecturas obtenidas y almacena temporalmente los valores para su posterior visualización en la pantalla OLED y su transmisión mediante tecnología LoRa.

En caso de que algún sensor no entregue una lectura válida, el firmware mantiene la continuidad del proceso de adquisición utilizando los mecanismos de validación implementados en el código, evitando que una lectura errónea interrumpa el funcionamiento general del nodo.

6.5 Asignación de pines de los sensores

La comunicación entre la placa Heltec WiFi LoRa 32 V3 y los sensores ambientales se realiza mediante las interfaces I2C, UART, I2S y entradas digitales, según las características de cada dispositivo. La asignación de pines utilizada en el firmware se mantiene constante para ambos nodos sensores y debe conservarse al realizar modificaciones o al incorporar nuevos dispositivos.

Tabla 11

Conexión de pines entre cada sensor y la placa Heltec

Sensor	Variable medida	Interfaz	Conexión (pines Heltec)
SHT31	Temperatura y humedad	I2C	SDA → GPIO41 SCL → GPIO42
BH1750	Iluminación (lux)	I2C	SDA → GPIO41 SCL → GPIO42 (mismo bus que SHT31)
MH-Z19C	CO ₂ (ppm)	UART	RX (Heltec) → GPIO6 TX (Heltec) → GPIO7
HC-SR501 (PIR)	Movimiento / presencia	Digital	OUT → GPIO4
INMP441	Nivel de ruido (dB)	I2S	SCK → GPIO47 WS → GPIO48 SD → GPIO5 L/R → GND

Nota. Los sensores SHT31 y BH1750 comparten el mismo bus I2C, mientras que el sensor MH-Z19C utiliza una interfaz UART dedicada y el micrófono INMP441 emplea la interfaz I2S para la captura de audio. Cualquier modificación de esta asignación deberá reflejarse en las definiciones correspondientes del firmware para garantizar el correcto funcionamiento del sistema.

Figura 19

Fragmento del firmware donde se definen los pines utilizados por los sensores conectados a la placa Heltec WiFi LoRa 32 V3

```
</> C++

// ===== Bus I2C =====
#define SDA_SENSORES 41
#define SCL_SENSORES 42

// ===== Sensor CO2 (MH-Z19C) =====
#define CO2_RX 6
#define CO2_TX 7

// ===== Sensor de movimiento (PIR) =====
#define PIR_PIN 4

// ===== Micrófono INMP441 (I2S) =====
#define I2S_SD_PIN 5
#define I2S_SCK_PIN 47
#define I2S_WS_PIN 48
```

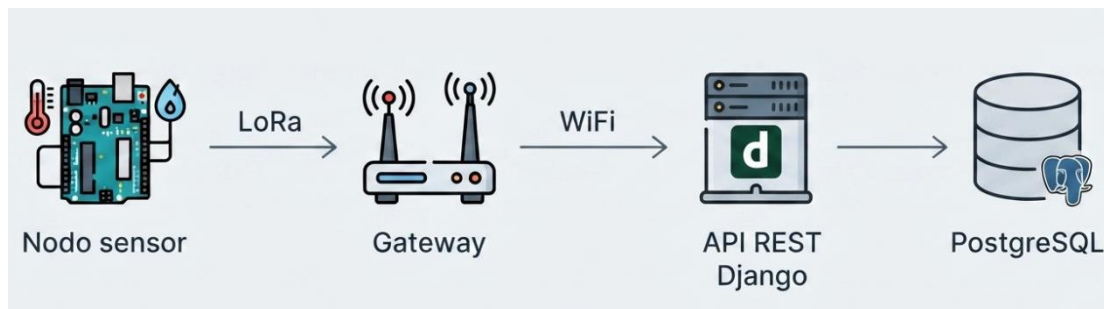
6.6 Firmware del gateway de pruebas

Durante la etapa de validación del sistema se implementó un firmware específico para un nodo gateway, cuya función consistió en recibir las tramas transmitidas mediante tecnología LoRa por los nodos sensores y reenviarlas al backend desarrollado en Django mediante una conexión WiFi.

A diferencia de los nodos sensores, el gateway no realiza mediciones ambientales. Su función se limita a actuar como intermediario entre la red LoRa y la red IP, permitiendo que la información recibida sea enviada a la API REST del sistema para su almacenamiento en la base de datos PostgreSQL.

Figura 20

Flujo de funcionamiento del gateway de pruebas



Nota. El gateway recibe las tramas LoRa enviadas por los nodos sensores, verifica la información recibida y la transmite al backend mediante una solicitud HTTP POST. Elaboración propia.

Paso 1. Configuración del gateway de prueba

Antes de compilar el firmware del gateway es necesario verificar los parámetros de configuración relacionados con la comunicación inalámbrica y el acceso al servidor donde se ejecuta el backend.

Tabla 12

Parámetros de configuración del gateway de pruebas

Parámetro	Descripción
SSID	Nombre de la red WiFi utilizada por el gateway.
PASSWORD	Contraseña de la red inalámbrica.
DJANGO_URL	Dirección del servicio REST donde se enviarán las mediciones recibidas.
Frecuencia LoRa	Debe coincidir con la utilizada por los nodos sensores (915 MHz).
Ancho de banda, SF, CR y Sync Word	Deben mantenerse iguales a los configurados en los nodos sensores para garantizar la comunicación.

La dirección definida en DJANGO_URL debe apuntar al endpoint implementado en el backend para la recepción de mediciones ambientales. Cualquier modificación en la dirección

IP del servidor o en el puerto de ejecución requiere actualizar este parámetro antes de compilar nuevamente el firmware.

Paso 2. Recepción y reenvío de datos

Una vez inicializado, el gateway permanece en espera de nuevas transmisiones LoRa. Cuando recibe una trama válida, extrae el contenido del paquete y realiza una solicitud HTTP POST al backend utilizando la red WiFi configurada.

El proceso ejecutado por el gateway puede resumirse en las siguientes etapas:

1. Inicialización del módulo LoRa y de la conexión WiFi.
2. Espera de paquetes transmitidos por los nodos sensores.
3. Recepción de la trama LoRa.
4. Envío de la información al backend mediante una solicitud HTTP POST.
5. Recepción del código de respuesta emitido por la API REST.

La utilización de este gateway permitió validar el funcionamiento completo del sistema durante las pruebas de integración, verificando la comunicación entre los nodos sensores, el backend desarrollado en Django y la base de datos PostgreSQL.

7. Solución de Problemas

Esta sección reúne las incidencias más comunes que pueden presentarse durante la compilación, ejecución y mantenimiento del sistema, así como las acciones recomendadas para su diagnóstico y corrección. Los problemas descritos corresponden a situaciones identificadas durante el desarrollo y las pruebas de integración del proyecto.

Tabla 13

Problemas frecuentes y solución recomendada

Problema	Posible causa	Solución
El nodo sensor no transmite datos mediante LoRa.	Los parámetros de comunicación (frecuencia, ancho de banda, Spreading Factor, Coding Rate o Sync Word) no coinciden entre el nodo sensor y el gateway.	Verificar que ambos dispositivos utilicen la misma configuración de comunicación LoRa.
El gateway recibe las tramas LoRa, pero los	El gateway no dispone de conexión WiFi o la dirección	Verificar la conexión a la red WiFi y confirmar la dirección IP, puerto

Problema	Posible causa	Solución
datos no aparecen en el dashboard.	configurada en DJANGO_URL es incorrecta.	y endpoint configurados en el firmware del gateway.
El backend no almacena las mediciones recibidas.	Las migraciones no fueron aplicadas o PostgreSQL no se encuentra en ejecución.	Ejecutar python manage.py migrate y comprobar que el servicio de PostgreSQL esté disponible.
Error HTTP durante el envío de datos al backend.	El servidor Django no está en ejecución o el endpoint configurado es incorrecto.	Confirmar que el backend se encuentre ejecutándose y verificar la configuración de DJANGO_URL.
Django no puede establecer conexión con PostgreSQL.	Error en las credenciales o parámetros definidos en el archivo .env.	Revisar los valores de DB_NAME, DB_USER, DB_PASSWORD, DB_HOST y DB_PORT.
La pantalla OLED permanece en blanco después del inicio.	Error en la inicialización de la librería heltec_unofficial o problema de alimentación del dispositivo.	Comprobar la alimentación de la placa y revisar los mensajes mostrados en el Monitor Serie durante la inicialización.
Un sensor no entrega lecturas válidas.	Conexión incorrecta, fallo de inicialización o problema de alimentación del sensor.	Verificar el cableado, la alimentación y la correcta inicialización del sensor correspondiente antes de iniciar el ciclo de monitoreo.

8. Mantenimiento, Escalabilidad y Seguridad

Este capítulo presenta recomendaciones orientadas al mantenimiento del sistema, la incorporación de nuevos nodos sensores y la protección de la información utilizada durante la ejecución del backend. Estas consideraciones facilitan la continuidad del desarrollo y la adaptación del sistema a futuras ampliaciones.

8.1 Mantenimiento

Las actividades de mantenimiento permiten conservar el correcto funcionamiento del sistema y facilitar futuras modificaciones tanto en el firmware como en el backend.

Tabla 14

Buenas prácticas recomendadas para el mantenimiento del sistema

Práctica	Descripción
Control de versiones	Gestionar el código fuente mediante Git utilizando confirmaciones descriptivas que permitan identificar y recuperar versiones estables del proyecto.
Depuración del firmware	Utilizar el Monitor Serie de Arduino IDE para verificar la inicialización de los sensores, la comunicación LoRa, la conexión WiFi del gateway y detectar posibles errores durante las pruebas del sistema. En la versión final se recomienda mantener únicamente los mensajes necesarios para el diagnóstico del funcionamiento.
Verificación de sensores	Comprobar periódicamente el funcionamiento de los sensores y contrastar sus lecturas con instrumentos de referencia cuando sea necesario.
Respaldo de la base de datos	Realizar copias de seguridad periódicas de PostgreSQL para preservar el historial de mediciones almacenadas.
Actualización de dependencias	Verificar la compatibilidad de las librerías utilizadas en Arduino IDE y de los paquetes del backend antes de realizar actualizaciones del proyecto.

8.2 Escalabilidad del sistema

La arquitectura implementada permite ampliar el sistema sin modificar su funcionamiento general. Para incorporar **nuevos nodos sensores** únicamente es necesario utilizar el mismo firmware base y asignar un identificador (NODO_ID) y un aula (AULA_ID) diferentes para cada dispositivo.

De igual forma, el sistema admite la incorporación de nuevas variables ambientales mediante la integración de sensores adicionales. Para ello es necesario actualizar el firmware del nodo correspondiente, incorporar el nuevo campo al modelo MedicionAmbiental y ejecutar las migraciones del backend para reflejar los cambios en la base de datos.

Cuando se incremente el número de nodos o se reduzca el intervalo de transmisión, se recomienda ajustar la temporización de envío con el fin de disminuir la probabilidad de colisiones durante la comunicación LoRa.

8.3 Seguridad y respaldo

El backend del sistema utiliza un archivo **.env** para almacenar la información sensible de configuración, el cual es cargado mediante la librería python-dotenv. En este archivo se definen la clave secreta de Django (SECRET_KEY) y los parámetros de conexión a la base de datos PostgreSQL (DB_NAME, DB_USER, DB_PASSWORD, DB_HOST y DB_PORT). Para evitar la exposición de esta información, el archivo .env se encuentra excluido del repositorio mediante el archivo .gitignore.

Con el fin de preservar la seguridad y disponibilidad del sistema, se recomienda considerar las siguientes prácticas:

- Utilizar contraseñas seguras para la base de datos PostgreSQL y para la cuenta administradora de Django.
- Mantener el archivo .env fuera del repositorio y no incluir información sensible directamente en el código fuente.
- Configurar el parámetro DEBUG = False cuando el sistema sea desplegado fuera del entorno de desarrollo, evitando la visualización de información de depuración ante posibles errores.
- Definir adecuadamente el parámetro ALLOWED_HOSTS, permitiendo únicamente el acceso desde las direcciones IP o dominios autorizados.
- Realizar copias de seguridad periódicas de la base de datos PostgreSQL para preservar el historial de mediciones y facilitar la recuperación de la información ante fallos del sistema.